

Big Data Analytics & Visualization

LAB 5: Hands-on with Hadoop YARN

Understanding Resource Management and Scheduling in YARN

Prepared by:

BOUHOUIA Yousra G1

BENHAMOUCHE Lydia G1

Academic Year 2024/2025

Description

In this hands-on lab, we explored the internal workings of Hadoop YARN (Yet Another Resource Negotiator). The focus was on understanding how YARN manages resources, schedules tasks, and monitors distributed applications. We executed MapReduce jobs, analyzed resource allocation using the ResourceManager UI, and observed scheduling behavior under different conditions.

Part 1: YARN Web Interface Exploration

The YARN ResourceManager web interface was accessed through the browser. We explored different sections including running and completed applications, scheduler details, nodes, and cluster metrics.

Observations

- All applications were in the **FINISHED** state.
- The scheduler in use was the **Capacity Scheduler**.
- The cluster consisted of **5 active nodes**.
- Total cluster resources included **40 GB memory** and **40 vCores**.

Answers to Questions

1. Number of containers currently allocated:

The number of running containers was **0**, indicating that no jobs were currently executing.

2. Total memory available:

The total available memory in the cluster was **40 GB**.

3. Queue names and capacities:

The system used a single queue named **default**, with approximately **100% capacity**, meaning it utilized the entire cluster resources.

Conclusion

From this part, we observed that YARN efficiently tracks application states and resource usage. The interface provides clear insights into cluster health, resource availability, and scheduling behavior.

Part 2: Running a Sample MapReduce Job

In this part, a sample MapReduce job (WordCount) was executed using the Hadoop examples package. The objective was to submit a job, monitor its execution through YARN, and analyze the output.

Job Execution

After executing the MapReduce WordCount job, the output was successfully generated and stored in HDFS.

The results were retrieved and verified, confirming correct processing of the input data.

```
student207@hadoop-master:~$ hdfs dfs -ls /user/student207/output
hdfs dfs -cat /user/student207/output/part-r-00000
Found 2 items
-rw-r--r--  3 student207 hadoop_students      0 2026-04-14 14:18 /user/student207/output/_SUCCESS
-rw-r--r--  3 student207 hadoop_students    53 2026-04-14 14:18 /user/student207/output/part-r-00000
analytics      2
big            1
data          1
hadoop        1
mapreduce      1
yarn          1
student207@hadoop-master:~$ |
```

Figure 1: HDFS Output of WordCount Job

This confirms that the MapReduce job correctly counted the occurrences of each word in the input file.

Monitoring via YARN ResourceManager

The MapReduce job was monitored using the YARN ResourceManager web interface.

- Application ID: application_1770582555454_0375
- Application Name: word count
- Queue: default
- State: FINISHED
- Final Status: SUCCEEDED

This confirms that the job was successfully executed and completed.

application_1770582555454_0375	student207	word count	MAPREDUCE	default	0	Tue Apr 14 15:16:35 +0100 2026	Tue Apr 14 15:16:35 +0100 2026	Tue Apr 14 15:16:46 +0100 2026	FINISHED	SUCCEEDED
--------------------------------	------------	------------	-----------	---------	---	--------------------------------	--------------------------------	--------------------------------	----------	-----------

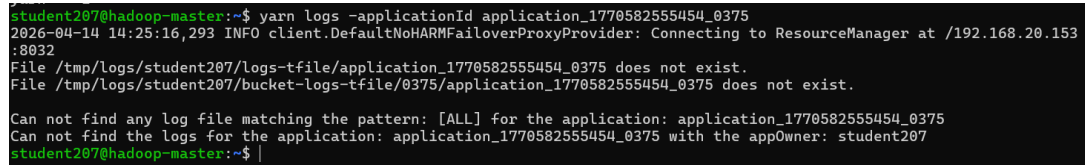
Figure 2: YARN UI with job progress

Log Analysis

The following command was used to retrieve application logs:

```
yarn logs -applicationId application_1770582555454_0375
```

However, no logs were returned. The system indicated that no matching log files were found for the specified application.



```
student207@hadoop-master:~$ yarn logs -applicationId application_1770582555454_0375
2026-04-14 14:25:16,293 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /192.168.20.153:8032
File /tmp/logs/student207/logs-tfile/application_1770582555454_0375 does not exist.
File /tmp/logs/student207/bucket-logs-tfile/0375/application_1770582555454_0375 does not exist.
Can not find any log file matching the pattern: [ALL] for the application: application_1770582555454_0375
Can not find the logs for the application: application_1770582555454_0375 with the appOwner: student207
student207@hadoop-master:~$
```

Figure 3: Log retrieval attempt output

Observation

The absence of logs suggests that YARN log aggregation may not be enabled or logs were not retained in the system. Despite this, the job status shown in the ResourceManager UI confirms successful execution.

Conclusion

In this part, we successfully executed a MapReduce WordCount job, verified its output, and monitored its execution through the YARN interface. This demonstrated how YARN manages and tracks distributed applications in a Hadoop environment.

Part 3: Resource Allocation Analysis

In this part, the resource allocation of the executed MapReduce application was analyzed using YARN tools and cluster information.

Application Overview

The application named **word count** was executed in the **default queue** and completed successfully with a final state of **SUCCEEDED** and a progress of **100%**.

```
student207@hadoop-master:~$ yarn application -list
2026-04-14 14:36:20,521 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /192.168.20.153:8032
Total number of applications (application-types: [], states: [SUBMITTED, ACCEPTED, RUNNING] and tags: []):0
  State      Application-Id      Application-Name      Application-Type      User      Queue
  Final-State      Progress      Tracking-URL
```

Figure 4: YARN Application List Showing No Running Jobs

Application Details

Detailed information about the application shows that the **ApplicationMaster** was executed on the node **hadoop-slave4**. The job tracking URL and execution metadata confirm successful completion.

```
student207@hadoop-master:~$ yarn application -status application_1770582555454_0375
2026-04-14 14:38:11,323 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /192.168.20.153:8032
2026-04-14 14:38:11,506 INFO conf.Configuration: resource-types.xml not found
2026-04-14 14:38:11,507 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
Application Report :
  Application-Id : application_1770582555454_0375
  Application-Name : word count
  Application-Type : MAPREDUCE
  User : student207
  Queue : default
  Application Priority : 0
  Start-Time : 1776176315115
  Finish-Time : 1776176328438
  Progress : 100%
  State : FINISHED
  Final-State : SUCCEEDED
  Tracking-URL : http://hadoop-slave4:19888/jobhistory/job/job_1770582555454_0375
  RPC Port : 45367
  AM Host : hadoop-slave4
  Aggregate Resource Allocation : 46912 MB-seconds, 25 vcore-seconds
  Aggregate Resource Preempted : 0 MB-seconds, 0 vcore-seconds
  Log Aggregation Status : DISABLED
  Diagnostics :
  Unmanaged Application : false
  Application Node Label Expression : <Not set>
  AM container Node Label Expression : <DEFAULT_PARTITION>
  TimeoutType : LIFETIME ExpiryTime : UNLIMITED RemainingTime : -1seconds
```

Figure 5: Application Status and Resource Allocation Details

Resource Usage

The total resources consumed by the application were:

- **Memory:** 46912 MB-seconds
- **CPU:** 25 vCore-seconds

This indicates the total amount of memory and CPU used during job execution.

Cluster Information

The cluster consists of **5 active nodes**, all in the **RUNNING** state:

- hadoop-slave1
- hadoop-slave2
- hadoop-slave3
- hadoop-slave4
- hadoop-master

```
student207@hadoop-master:~$ yarn node -list
2026-04-14 14:38:22,040 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at /192.168.20.153:8032
Total Nodes:5


| Node-Id             | Node-State | Node-Http-Address  | Number-of-Running-Containers |
|---------------------|------------|--------------------|------------------------------|
| hadoop-slave1:37857 | RUNNING    | hadoop-slave1:8042 | 0                            |
| hadoop-slave4:45611 | RUNNING    | hadoop-slave4:8042 | 0                            |
| hadoop-slave2:39151 | RUNNING    | hadoop-slave2:8042 | 0                            |
| hadoop-master:35645 | RUNNING    | hadoop-master:8042 | 0                            |
| hadoop-slave3:39149 | RUNNING    | hadoop-slave3:8042 | 0                            |


student207@hadoop-master:~$
```

Figure 6: Cluster Nodes Status and Resource Availability

At the time of observation, all nodes had **0 running containers**, indicating that the job had already finished execution.

Observations

- The application completed successfully with no errors.
- The ApplicationMaster was executed on **hadoop-slave4**.
- Resource consumption was moderate and efficiently managed.
- Log aggregation was **disabled**, explaining the absence of logs.

Conclusion

The MapReduce job was successfully executed on the YARN cluster. The system efficiently allocated resources across the 5 available nodes, demonstrating YARN's capability in managing distributed applications and resource scheduling.

Part 4: Simulation of Resource Constraints

In this part, two MapReduce WordCount jobs were executed almost simultaneously using different inputs. The execution of both jobs was monitored using the YARN Resource-Manager interface.

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus
application_1770582555454_0387	student207	word count	MAPREDUCE		default	0	Tue Apr 14 15:59:26 +0100 2026	Tue Apr 14 15:59:27 +0100 2026	Tue Apr 14 15:59:40 +0100 2026	FINISHED	SUCCEEDED
application_1770582555454_0386	student207	word count	MAPREDUCE		default	0	Tue Apr 14 15:59:25 +0100 2026	Tue Apr 14 15:59:25 +0100 2026	Tue Apr 14 15:59:39 +0100 2026	FINISHED	SUCCEEDED

Figure 7: YARN UI Showing Two Concurrent MapReduce Jobs

Observations

- Two applications were executed concurrently.
- Both jobs completed successfully with state **FINISHED** and final status **SUCCEEDED**.
- Job 1 started at **15:59:25** and finished at **15:59:39**.
- Job 2 started at **15:59:26** and finished at **15:59:40**.

Answers to Questions

Are containers queued?

No significant queuing was observed. Both jobs started almost at the same time (within one second) and immediately moved to execution without waiting.

Does one job delay the other?

No noticeable delay occurred. Both jobs executed concurrently and finished nearly at the same time, with only about one second difference.

Conclusion

The results indicate that the cluster had sufficient resources to handle multiple jobs simultaneously. YARN efficiently distributed resources, allowing parallel execution without queuing or delay.

Part 5: Scheduling Policy Mini-Experiment

In this part, the objective was to modify the scheduler configuration to create multiple queues and observe their behavior.

Remark

The scheduling policy experiment could not be fully executed because the Hadoop configuration files are owned by the **hadoop** user and are not writable by the current user (**student207**). Therefore, it was not possible to modify the scheduler configuration or restart YARN services.

Expected Behavior

- Jobs submitted to different queues would share cluster resources based on predefined capacities.
- Each queue would receive a fixed portion of resources (e.g., 50% each).
- If one queue is idle, its unused resources could be utilized by other active queues.

Part 6: Wrap-up and Discussion

What did you observe about resource sharing?

YARN dynamically allocates resources among applications, allowing multiple jobs to run concurrently while efficiently sharing cluster resources.

How does YARN improve over Hadoop 1?

YARN separates resource management from job execution, improving scalability and flexibility. Unlike Hadoop 1, which was limited to MapReduce, YARN supports multiple processing frameworks.

What would happen in a multi-tenant environment?

YARN manages multiple users by allocating resources fairly using queues and scheduling policies, ensuring balanced resource usage and preventing any single user from monopolizing the cluster.

Conclusion

This lab demonstrated how Hadoop YARN manages distributed resources and schedules jobs efficiently. Through practical experiments, we observed job execution, resource allocation, and concurrent processing. These results highlight the importance of YARN in managing large-scale data processing systems and ensuring efficient utilization of cluster resources.